



UNIVERSIDAD DE CARABOBO
FACULTAD DE INGENIERIA
MAESTRIA EN MATEMÁTICAS Y COMPUTACION

Asignación Dos

Fundamentos y Algoritmos de Machine Learning

Bases Matematicas y ejemplos de codigo

Autor: Ing. Melvin E. Padilla
Cátedra: Diseño de algoritmos
Profesor: Alejandro Bolivar

Infraestructura de Cómputo:

Lenovo IdeaPad Gaming 3 | AMD Ryzen™ 7 4800H (16 Threads)

Ubuntu 24.04.4 LTS | Spyder IDE

Valencia, Mayo de 2026

Índice

1. Aprendizaje Supervisado (Supervised Learning)	2
1.1. Algoritmos de Regresión	2
1.2. Algoritmos de Clasificación	2
2. Aprendizaje No Supervisado (Unsupervised Learning)	3
2.1. Algoritmos de Agrupamiento (Clustering)	3
2.2. Algoritmos de Reducción de Dimensionalidad	3
2.3. Asociación	3
3. Aprendizaje por Refuerzo (Reinforcement Learning)	4
4. Métodos de Ensamble (Ensemble Methods)	4
5. Aprendizaje Profundo (Deep Learning)	4
6. Bases Matemáticas de los Algoritmos	5
6.1. 1. Álgebra Lineal: La Representación Espacial	5
6.2. 2. Cálculo Diferencial y Optimización Continua	6
6.3. 3. Probabilidad y Estadística	6
6.4. 4. Teoría de la Información y Distancias Métricas	7
7. Implementación en Código	8
7.1. 1. Regresión Lineal: Optimización por Descenso del Gradiente	8
7.2. 2. K-Vecinos Más Cercanos (KNN): Espacios Métricos	9
7.3. 3. K-Means: Agrupamiento no Supervisado	10
8. Red Neuronal (Perceptrón Multicapa) desde Cero	12
8.1. El Reto Matemático: Backpropagation y Regla de la Cadena	12
8.2. Implementación Paso a Paso	12
8.3. Resumen del Flujo Matemático en Código	18

El universo del Machine Learning (Aprendizaje Automático) es enorme, pero para entenderlo de forma clara, los algoritmos se agrupan principalmente según cómo aprenden de los datos (el tipo de supervisión) y la tarea específica que realizan.

A continuación mostramos un listado estructurado de los tipos de algoritmos más utilizados:

1. Aprendizaje Supervisado (Supervised Learning)

En este tipo, el algoritmo aprende a partir de un conjunto de datos de entrenamiento que ya incluye las respuestas correctas (etiquetas o labels). Su objetivo es encontrar la función matemática que mapea las entradas con las salidas.

1.1. Algoritmos de Regresión

Se usan cuando el resultado que queremos predecir es un valor numérico continuo (como el precio de una casa, la temperatura o la demanda de un producto).

- **Regresión Lineal (Linear Regression):** Modela la relación entre variables mediante una línea recta.
- **Regresión Polinomial:** Para relaciones no lineales más complejas.
- **Vectores de Soporte para Regresión (SVR):** Adapta el concepto de margen de SVM para variables continuas.

1.2. Algoritmos de Clasificación

Se usan cuando la salida es una categoría o clase (como detectar si un correo es Spam/No Spam, o si un cliente va a cancelar una suscripción).

- **Regresión Logística (Logistic Regression):** A pesar de su nombre, se usa para clasificación binaria calculando probabilidades.
- **K-Vecinos Más Cercanos (K-Nearest Neighbors - KNN):** Clasifica un punto según las clases de los puntos más cercanos a él en el espacio.
- **Máquinas de Vectores de Soporte (Support Vector Machines - SVM):** Encuentra el hiperplano óptimo que maximiza la separación entre diferentes clases.
- **Árboles de Decisión (Decision Trees):** Modelos basados en reglas de "si/entonces" que dividen los datos de forma jerárquica.

- **Naive Bayes:** Basado en el teorema de Bayes; asume que las variables predictoras son independientes entre sí.

2. Aprendizaje No Supervisado (Unsupervised Learning)

Aquí los datos no tienen etiquetas. El algoritmo no sabe la respuesta correcta"; su trabajo es explorar la estructura de los datos para encontrar patrones, regularidades o agrupaciones ocultas de forma autónoma.

2.1. Algoritmos de Agrupamiento (Clustering)

Dividen el conjunto de datos en grupos (clusters) donde los elementos de un mismo grupo son muy similares entre sí y diferentes a los de otros grupos.

- **K-Means:** Agrupa los datos en K clusters asignando cada punto al centroide más cercano.
- **DBSCAN:** Agrupamiento basado en la densidad de los puntos; ideal para encontrar formas complejas y detectar ruido/outliers.
- **Agrupamiento Jerárquico (Hierarchical Clustering):** Construye una estructura de árbol (dendrograma) para agrupar los datos de forma ascendente o descendente.

2.2. Algoritmos de Reducción de Dimensionalidad

Se usan cuando tenemos demasiadas variables (columnas) y queremos simplificar el dataset manteniendo la mayor cantidad de información posible.

- **Análisis de Componentes Principales (PCA):** Transforma variables correlacionadas en un conjunto menor de variables lineales no correlacionadas (componentes principales).
- **t-SNE / UMAP:** Técnicas no lineales excelentes para reducir dimensiones complejas a 2D o 3D con el fin de visualizarlas.

2.3. Asociación

- **Algoritmo Apriori:** Descubre reglas de asociación en bases de datos (por ejemplo, "el 80 % de las personas que compran leche también compran pan").

3. Aprendizaje por Refuerzo (Reinforcement Learning)

Este enfoque no usa datos estáticos. En su lugar, un agente aprende a tomar decisiones en un entorno interactivo mediante un sistema de recompensas y castigos por medio del ensayo y error.

- **Q-Learning:** Un algoritmo basado en tablas que aprende el valor de tomar una acción en un estado determinado.
- **Deep Q-Networks (DQN):** Combina el aprendizaje por refuerzo con redes neuronales profundas para manejar entornos de altísima complejidad (como videojuegos o robótica).

4. Métodos de Ensamble (Ensemble Methods)

No son un tipo de aprendizaje diferente, sino una estrategia poderosa: combinan múltiples modelos individuales (generalmente árboles de decisión) para formar un modelo mucho más robusto y preciso.

- **Random Forest:** Un ensamble de muchos árboles de decisión entrenados en diferentes subdivisiones de los datos (Bagging). Reducen drásticamente el sobreajuste (overfitting).
- **Gradient Boosting Machines (GBM):** Entrena árboles de forma secuencial, donde cada nuevo árbol corrige los errores del anterior (Boosting).
- **XGBoost / LightGBM / CatBoost:** Implementaciones ultra optimizadas y de alto rendimiento del Gradient Boosting, muy populares en la ciencia de datos aplicada.

5. Aprendizaje Profundo (Deep Learning)

Es un subcampo del machine learning que se basa en Redes Neuronales Artificiales con muchas capas ocultas. Son los algoritmos estándar para manejar datos no estructurados como imágenes, audio y texto.

- **Redes Neuronales Densas (Multi-Layer Perceptron - MLP):** La estructura básica donde todas las neuronas de una capa se conectan con la siguiente.

- **Redes Neuronales Convolucionales (CNN):** Especializadas en el procesamiento de imágenes y visión artificial (detección de patrones espaciales).
- **Redes Neuronales Recurrentes (RNN) / LSTM:** Diseñadas para datos secuenciales o temporales (como texto o series de tiempo).
- **Transformers:** La arquitectura moderna que revolucionó el Procesamiento del Lenguaje Natural (NLP), basada en mecanismos de atención (la base de los modelos de lenguaje actuales).

Nota de aplicación: La elección del algoritmo depende completamente de la naturaleza de los datos (si están etiquetados o no, si son numéricos, categóricos o secuenciales) y del costo computacional que se pueda permitir para el entrenamiento. El Machine Learning no es más que matemáticas aplicadas al reconocimiento de patrones y la aproximación de funciones. Es importante entender que bajo el «capó» de estos algoritmos operan cuatro pilares matemáticos fundamentales: Álgebra Lineal, Cálculo Diferencial (Optimización), Probabilidad/Estadística y Teoría de la Información.

6. Bases Matemáticas de los Algoritmos

6.1. 1. Álgebra Lineal: La Representación Espacial

El álgebra lineal proporciona las estructuras de datos (vectores, matrices, tensores) y las operaciones para proyectar y transformar el espacio de las características (feature space).

Regresión Lineal y Múltiple: El modelo se define como $\hat{y} = X\beta$, donde $X \in \mathbb{R}^{m \times n}$ es la matriz de diseño y $\beta \in \mathbb{R}^n$ es el vector de pesos. La solución analítica exacta (Ecuación Normal) que minimiza el error cuadrático medio se obtiene mediante pseudoinversa:

$$\beta = (X^T X)^{-1} X^T y$$

Análisis de Componentes Principales (PCA): Se basa en el Teorema Espectral. Dada una matriz de datos centrada X , calculamos su matriz de covarianza $\Sigma = \frac{1}{m-1} X^T X$. PCA encuentra los autovectores (v) y autovalores (λ) resolviendo la ecuación característica:

$$\Sigma v = \lambda v$$

Los autovectores correspondientes a los mayores autovalores representan las direcciones de máxima varianza, permitiendo proyectar los datos a un subespacio de menor dimensión $\mathbb{R}^k$ ($k < n$).

6.2. 2. Cálculo Diferencial y Optimización Continua

El cálculo vectorial es el motor del «aprendizaje». Los algoritmos formulan una función de pérdida (Loss Function) $L(\theta)$ y buscan sus mínimos globales o locales en espacios n-dimensionales.

Descenso del Gradiente (Gradient Descent): Usado en Regresión, Redes Neuronales y Logistic Regression. Se actualizan los parámetros iterativamente en la dirección opuesta al gradiente de la función de pérdida respecto a los pesos θ :

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

donde η es la tasa de aprendizaje (learning rate).

Backpropagation en Deep Learning (MLP, CNN, RNN): No es más que la aplicación sistemática de la Regla de la Cadena del cálculo multivariable para derivar el error a través de múltiples capas de funciones compuestas. Para un peso w_{ij} en una capa oculta, el gradiente es:

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial a_j} \frac{\partial a_j}{\partial z_j} \frac{\partial z_j}{\partial w_{ij}}$$

Máquinas de Vectores de Soporte (SVM): Utilizan optimización convexa restringida. El objetivo es maximizar el margen $2/\|w\|$, lo que equivale a minimizar $\frac{1}{2}\|w\|^2$ sujeto a restricciones lineales $y_i(w^T x_i + b) \geq 1$. Esto se resuelve utilizando Multiplicadores de Lagrange y las condiciones de Karush-Kuhn-Tucker (KKT), llevándolo a su forma dual, que permite aplicar el «Kernel Trick» mapeando datos a dimensiones infinitas sin costo computacional directo:

$$\max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

6.3. 3. Probabilidad y Estadística

Permiten modelar la incertidumbre y realizar inferencias sobre distribuciones subyacentes.

Regresión Logística: Modela la probabilidad de que una muestra pertenezca a la clase 1 usando la función sigmoide:

$$P(y = 1|x; w) = \sigma(w^T x) = \frac{1}{1 + e^{-w^T x}}$$

El modelo aprende maximizando la verosimilitud de los datos (Maximum Likelihood Estimation - MLE), lo que matemáticamente equivale a minimizar la pérdida de entropía cruzada binaria (Binary Cross-Entropy).

Naive Bayes: Se fundamenta directamente en el Teorema de Bayes, asumiendo ingenuamente que las características x_i son condicionalmente independientes dada la clase y :

$$P(y|x_1, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i|y)$$

K-Means (Clustering): Desde una perspectiva estadística, K-Means es un caso especial y determinista de un Gaussian Mixture Model (GMM) asumiendo matrices de covarianza esféricas y varianzas idénticas. Minimiza la inercia (suma de cuadrados dentro del cluster):

$$J = \sum_{j=1}^K \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

6.4. 4. Teoría de la Información y Distancias Métricas

Estos conceptos rigen los algoritmos basados en topología espacial y sistemas de reglas.

Árboles de Decisión (Decision Trees) y Random Forest: Utilizan la Entropía de Shannon para medir la impureza de un nodo. La entropía de un conjunto S con clases de proporciones p_i es:

$$H(S) = - \sum_i p_i \log_2(p_i)$$

El algoritmo elige las divisiones de los datos maximizando la Ganancia de Información (Information Gain), que es la reducción de la entropía tras dividir el conjunto de datos por un atributo A :

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

K-Vecinos Más Cercanos (KNN): Depende íntegramente de espacios métricos y funciones de distancia $d(x, y)$, comúnmente generalizadas por la Distancia de Minkowski:

$$D(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

Donde $p = 1$ es la distancia de Manhattan, y $p = 2$ es la distancia Euclidiana.

7. Implementación en Código

Construir estos algoritmos desde cero es el mejor ejercicio para asentar la teoría. Para esto, Python es ideal gracias a numpy, que permite traducir directamente las operaciones de álgebra lineal y cálculo matricial a código sin ocultar el «cómo» debajo de cajas negras prefabricadas.

A continuación se muestran tres algoritmos fundamentales (uno de regresión, uno de clasificación y uno de agrupamiento), codificados desde cero para entender su mecánica interna.

7.1. 1. Regresión Lineal: Optimización por Descenso del Gradiente

Este código muestra cómo se aplica el cálculo diferencial. En lugar de resolver la ecuación normal exacta, inicializamos pesos en cero y calculamos las derivadas parciales de la función de pérdida iterativamente para «descender» hacia el mínimo error.

```
1 import numpy as np
2
3 class RegresionLineal:
4     def __init__(self, tasa_aprendizaje=0.01, iteraciones
5         =1000):
6         # Hiperparámetros del modelo
7         self.lr = tasa_aprendizaje
8         self.iteraciones = iteraciones
9         self.pesos = None
10        self.sesgo = None
11
12    def entrenar(self, X, y):
13        muestras, características = X.shape
14
15        # Inicializamos los parámetros matemáticos (vector w
16        y escalar b)
17        self.pesos = np.zeros(características)
18        self.sesgo = 0
19
20        # Proceso de Optimización (Gradient Descent)
21        for _ in range(self.iteraciones):
22            # 1. Función de hipótesis lineal:  $y = Xw + b$ 
23            y_pred = np.dot(X, self.pesos) + self.sesgo
```

```

23         # 2. Cálculo de gradientes (Derivadas parciales
del Error Cuadrático Medio)
24         # dw = (1/N) * X^T * (y_pred - y)
25         dw = (1 / muestras) * np.dot(X.T, (y_pred - y))
26         # db = (1/N) * sum(y_pred - y)
27         db = (1 / muestras) * np.sum(y_pred - y)
28
29         # 3. Actualización de parámetros en dirección
opuesta al gradiente
30         self.pesos -= self.lr * dw
31         self.sesgo -= self.lr * db
32
33     def predecir(self, X):
34         return np.dot(X, self.pesos) + self.sesgo

```

7.2. 2. K-Vecinos Más Cercanos (KNN): Espacios Métricos

Este algoritmo no tiene una fase de «entrenamiento» como tal (es un modelo lazy). Su núcleo matemático es netamente geométrico: calcular distancias en un espacio n-dimensional.

```

1 import numpy as np
2 from collections import Counter
3
4 def distancia_euclidiana(x1, x2):
5     # Norma L2:  $D(x, y) = \sqrt{\sum |x_i - y_i|^2}$ 
6     return np.sqrt(np.sum((x1 - x2)**2))
7
8 class KNN:
9     def __init__(self, k=3):
10         self.k = k
11
12     def entrenar(self, X, y):
13         # Memorización pura del espacio espacial de datos
14         self.X_train = X
15         self.y_train = y
16
17     def predecir(self, X):
18         predicciones = [self._predecir_uno(x) for x in X]
19         return np.array(predicciones)
20
21     def _predecir_uno(self, x):

```

```

22         # 1. Calcular las distancias entre el punto 'x' y
    todo el dataset
23         distancias = [distancia_euclidiana(x, x_train) for
    x_train in self.X_train]
24
25         # 2. Ordenar por distancia y extraer los índices de
    los 'k' más cercanos
26         k_indices = np.argsort(distancias)[:self.k]
27
28         # 3. Consultar las etiquetas (clases) de esos vecinos
29         k_etiquetas = [self.y_train[i] for i in k_indices]
30
31         # 4. Inferencia por votación mayoritaria (moda)
32         mas_comun = Counter(k_etiquetas).most_common(1)
33         return mas_comun[0][0]

```

7.3. 3. K-Means: Agrupamiento no Supervisado

K-Means es un algoritmo iterativo puro. El código refleja la alternancia de dos pasos estadísticos: asignación de distancias mínimas y recálculo de la media geométrica (el nuevo centroide).

```

1 import numpy as np
2
3 class KMeans:
4     def __init__(self, K=3, max_iter=100):
5         self.K = K
6         self.max_iter = max_iter
7         self.centroides = []
8
9     def entrenar(self, X):
10         muestras, caracteristicas = X.shape
11
12         # 1. Inicialización: Elegir K puntos aleatorios como
    centroides iniciales
13         indices_aleatorios = np.random.choice(muestras, self.
    K, replace=False)
14         self.centroides = X[indices_aleatorios]
15
16         for _ in range(self.max_iter):
17             # 2. Asignación: Crear agrupaciones asociando
    cada punto a su centroide más cercano
18             clusters = self._crear_clusters(X)

```

```

19
20         # Guardamos el estado anterior para verificar la
convergencia
21         centroides_anteriores = self.centroides.copy()
22
23         # 3. Actualización: Mover el centroide a la media
matemática de su agrupación
24         self.centroides = self._obtener_nuevos_centroides
(clusters, X)
25
26         # 4. Condición de convergencia: Si los centroides
ya no se mueven, terminamos
27         if np.all(centroides_anteriores == self.
centroides):
28             break
29
30         return self._obtener_etiquetas_cluster(clusters, X)
31
32     def _crear_clusters(self, X):
33         # Inicializar listas vacías para cada cluster
34         clusters = [[] for _ in range(self.K)]
35         for idx, muestra in enumerate(X):
36             # Distancia euclidiana (norma vectorial) a todos
los centroides
37             distancias = [np.linalg.norm(muestra - c) for c
in self.centroides]
38             centroide_mas_cercano = np.argmin(distancias)
39             clusters[centroide_mas_cercano].append(idx)
40         return clusters
41
42     def _obtener_nuevos_centroides(self, clusters, X):
43         nuevos_centroides = np.zeros((self.K, X.shape[1]))
44         for idx_cluster, cluster in enumerate(clusters):
45             if len(cluster) == 0:
46                 continue # Proteccion contra clusters vacíos
47             # La nueva ubicación es el vector medio de los
puntos asignados
48             media_cluster = np.mean(X[cluster], axis=0)
49             nuevos_centroides[idx_cluster] = media_cluster
50         return nuevos_centroides
51
52     def _obtener_etiquetas_cluster(self, clusters, X):

```

```

53         # Aplanar la lista de agrupaciones a un vector
    unidimensional de etiquetas
54     etiquetas = np.empty(X.shape[0])
55     for idx_cluster, cluster in enumerate(clusters):
56         for idx_muestra in cluster:
57             etiquetas[idx_muestra] = idx_cluster
58     return etiquetas

```

8. Red Neuronal (Perceptrón Multicapa) desde Cero

Programar un Perceptrón Multicapa (MLP) desde cero es el «rito de iniciación» para entender realmente cómo aprenden los modelos de Deep Learning. De modo tal que al eliminar las librerías como TensorFlow o PyTorch, es obligatorio gestionar manualmente el flujo de datos y, lo más importante, las matemáticas del entrenamiento.

8.1. El Reto Matemático: Backpropagation y Regla de la Cadena

El objetivo de una red neuronal es minimizar una función de pérdida (Loss Function), comúnmente el Error Cuadrático Medio (MSE), que mide qué tan lejos están nuestras predicciones de los valores reales.

$$L = MSE = \frac{1}{2n} \sum (y_{pred} - y_{real})^2$$

El «aprendizaje» consiste en ajustar los pesos (W) y sesgos (b) de cada capa para que esta pérdida disminuya. Para saber cómo ajustarlos, necesitamos calcular el gradiente de la pérdida respecto a cada parámetro: $\frac{\partial L}{\partial W}$ y $\frac{\partial L}{\partial b}$.

Como la red es una composición secuencial de funciones (capa tras capa), usamos la Regla de la Cadena del cálculo multivariable para derivar hacia atrás (desde la salida hacia la entrada).

8.2. Implementación Paso a Paso

Se debe diseñar una clase en Python, en el caso siguiente, llamada `NeuralNetworkMLP` que gestionará una arquitectura flexible. Se utiliza en el ejemplo la función de activación Sigmoide por su simplicidad matemática al derivar.

0. Funciones Auxiliares (Activación y sus Derivadas)

Primero se defina la función Sigmoide $\sigma(z)$ y su derivada $\sigma'(z)$. Su derivada tiene

la propiedad elegante de definirse en términos de la propia salida de la sigmoide ($f(z)$).

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$$

1. Arquitectura de la Clase

La clase gestionará tres partes clave:

- Inicialización: Crear pesos y sesgos con valores aleatorios.
- Forward Propagation (Propagación Hacia Adelante): Calcular la salida de la red.
- Backpropagation (Retropropagación): Calcular los gradientes y actualizar parámetros.

2. Código Completo desde Cero

```
1 import numpy as np
2
3 # 0. Funciones Matemáticas Auxiliares
4 def sigmoide(z):
5     """Función de activación Sigmoide."""
6     return 1 / (1 + np.exp(-z))
7
8 def sigmoide_derivada(salida_sigmoide):
9     """
10     Derivada de la Sigmoide.
11     Ojo: Recibe la SALIDA de la sigmoide (ya activada), no la
12     entrada z.
13     La fórmula matemática es sigmoide(z) * (1 - sigmoide(z)).
14     """
15     return salida_sigmoide * (1 - salida_sigmoide)
16
17 class NeuralNetworkMLP:
18     def __init__(self, capas_arquitectura, tasa_aprendizaje
19                 =0.1):
20         """
21         Argumentos:
22             capas_arquitectura: Lista con el número de
23             neuronas por capa.
24                                     Ej: [2, 4, 1] -> 2 entradas, 4
25                                     ocultas, 1 salida.
26             tasa_aprendizaje: Coeficiente alpha para el
27             gradient descent.
```

```

23     """
24     self.arquitectura = capas_arquitectura
25     self.lr = tasa_aprendizaje
26     self.parametros = {} # Aquí guardaremos W y b
27     self.memoria = {}    # Aquí guardaremos activaciones
(A) y entradas netas (Z)
28
29     # 1. Inicialización de Parámetros
30     for l in range(1, len(capas_arquitectura)):
31         # Pesos  $W^{[l]}$ : matriz (neuronas_capa_actual,
neuronas_capa_anterior)
32         self.parametros['W' + str(l)] = np.random.randn(
33             capas_arquitectura[l], capas_arquitectura[l
-1]
34         ) * 0.1 # Escala pequeña
35
36         # Sesgos  $b^{[l]}$ : vector columna (
neuronas_capa_actual, 1)
37         self.parametros['b' + str(l)] = np.zeros((
capas_arquitectura[l], 1))
38
39     def forward(self, X):
40         """
41         Propagación hacia adelante.
42         Recibe X: datos de entrada de forma (características,
muestras).
43         Retorna la activación final (predicción).
44         """
45         # La 'activación' de la capa 0 es la entrada misma.
46         activacion_anterior = X
47         self.memoria['A0'] = X # Guardar en memoria para
backprop
48
49         num_capas = len(self.arquitectura) - 1
50
51         for l in range(1, num_capas + 1):
52             # Obtener W y b de la capa l
53             W = self.parametros['W' + str(l)]
54             b = self.parametros['b' + str(l)]
55
56             # Combinación lineal:  $Z^{[l]} = W^{[l]} * A^{[l-1]} + b$ 
 $^{[l]}$ 
57             entrada_neta = np.dot(W, activacion_anterior) + b

```

```

58         # Función de activación:  $A^{[l]} = \text{sigma}(Z^{[l]})$ 
59         activacion_actual = sigmoide(entrada_neta)
60
61         # Guardar en memoria para usar en backprop
62         self.memoria['Z' + str(l)] = entrada_neta
63         self.memoria['A' + str(l)] = activacion_actual
64
65         # Actualizar para la siguiente iteración
66         activacion_anterior = activacion_actual
67
68     return activacion_actual # La última activación es la
69     predicción Y_hat
70
71     def backward(self, Y_real):
72         """
73         Retropropagación y actualización de parámetros.
74         Recibe Y_real: etiquetas reales de forma (salidas,
75         muestras).
76         """
77         num_muestras = Y_real.shape[1]
78         num_capas = len(self.arquitectura) - 1
79         A_final = self.memoria['A' + str(num_capas)]
80
81         # --- A. Capa de Salida (L) ---
82         # 1. Error de la capa de salida (delta L).
83         # Proviene de derivar MSE respecto a  $Z^{[L]}$ :  $(Y_{\text{hat}} - Y) * \text{sigmoide}'(Z^{[L]})$ 
84         # Pero  $\text{sigmoide}'(Z) = A * (1 - A)$ , así que usamos
85         A_final.
86
87         # Ojo: dA (derivada pérdida vs activación final) es (
88         A_final - Y_real)
89         dZ_actual = (A_final - Y_real) * sigmoide_derivada(
90         A_final)
91
92         # 2. Gradientes de W y b para la capa de salida.
93         #  $dW^{[L]} = (1/m) * dZ^{[L]} * A^{[L-1]^T}$ 
94         activacion_anterior = self.memoria['A' + str(
95         num_capas - 1)]
96         dW_actual = (1 / num_muestras) * np.dot(dZ_actual,
97         activacion_anterior.T)
98         db_actual = (1 / num_muestras) * np.sum(dZ_actual,

```

```

axis=1, keepdims=True)
93
94     # 3. Guardar gradientes y actualizar dZ para la capa
anterior (Regla de la cadena)
95     gradientes = {}
96     gradientes['dW' + str(num_capas)] = dW_actual
97     gradientes['db' + str(num_capas)] = db_actual
98
99     # dZ para propagar hacia atrás:  $dZ^{[l-1]} = (W^{[l]T} * dZ^{[l]}) * \text{sigmoide}'(Z^{[l-1]})$ 
100     dZ_propagar = dZ_actual
101
102     # --- B. Capas Ocultas (L-1 hasta 1) ---
103     for l in range(num_capas - 1, 0, -1):
104         # dZ_propagar es  $dZ^{[l+1]}$  (capa siguiente)
105         # W_siguiente es  $W^{[l+1]}$ 
106         W_siguiente = self.parametros['W' + str(l+1)]
107
108         # Z de la capa actual
109         Z_actual = self.memoria['Z' + str(l)]
110         # A de la capa actual
111         A_actual = self.memoria['A' + str(l)]
112
113         # Calcular  $dZ^{[l]}$  (Error de la capa actual)
114         usando REGLA DE LA CADENA
115         # (Derivada del error respecto a la entrada neta
116         Z de esta capa)
117
118         #  $dZ^{[l]} = (W^{[l+1]T} * dZ^{[l+1]}) * \text{sigmoide}'(Z^{[l]})$ 
119         #  $\text{sigmoide}'(Z) = A * (1-A)$ 
120
121         error_propagado = np.dot(W_siguiente.T,
dZ_propagar)
122         derivada_activacion = sigmoide_derivada(A_actual)
123         dZ_actual = error_propagado * derivada_activacion
124
125         # Gradientes de W y b para esta capa oculta
126         #  $dW^{[l]} = (1/m) * dZ^{[l]} * A^{[l-1]T}$ 
127         activacion_capa_anterior = self.memoria['A' + str
(l-1)]
128         dW_actual = (1 / num_muestras) * np.dot(dZ_actual
, activacion_capa_anterior.T)

```

```

127         db_actual = (1 / num_muestras) * np.sum(dZ_actual
128         , axis=1, keepdims=True)
129
130         # Guardar gradientes
131         gradientes['dW' + str(l)] = dW_actual
132         gradientes['db' + str(l)] = db_actual
133
134         # dZ_propagar se convierte en dZ^[l] para la
135         siguiente iteración de bucle (capa l-1)
136         dZ_propagar = dZ_actual
137
138         # --- C. Actualización de Parámetros (Gradient
139         Descent) ---
140         for l in range(1, num_capas + 1):
141             # W^[l] = W^[l] - lr * dW^[l]
142             self.parametros['W' + str(l)] -= self.lr *
143             gradientes['dW' + str(l)]
144             # b^[l] = b^[l] - lr * db^[l]
145             self.parametros['b' + str(l)] -= self.lr *
146             gradientes['db' + str(l)]
147
148         def entrenar(self, X, Y, epochs):
149             """Bucle principal de entrenamiento."""
150             for epoch in range(epochs):
151                 # 1. Forward
152                 Y_pred = self.forward(X)
153
154                 # 2. Backward & Update
155                 self.backward(Y)
156
157                 # 3. Mostrar progreso opcionalmente (costo MSE)
158                 if epoch % 100 == 0:
159                     costo = np.mean(np.square(Y_pred - Y)) # MSE
160                     print(f"Época {epoch}: Costo MSE = {costo:.6f
161                     }")
162
163         def predecir(self, X):
164             """Hace inferencia usando forward."""
165             return self.forward(X)

```

3. Ejemplo de Uso: Resolviendo XOR

El problema XOR es clásico porque no puede resolverse con una simple regresión lineal o un perceptrón de una sola capa; requiere al menos una capa oculta.

```

1 # Datos XOR (4 muestras, 2 características)
2 # X: forma (características, muestras) -> (2, 4)
3 X_train = np.array([
4     [0, 0, 1, 1], # Característica 1
5     [0, 1, 0, 1]  # Característica 2
6 ])
7
8 # Etiquetas XOR (1 salida por muestra)
9 # Y: forma (salidas, muestras) -> (1, 4)
10 Y_train = np.array([[0, 1, 1, 0]])
11
12 # 1. Instanciar la red: [2 entradas, 3 ocultas, 1 salida]
13 mlp = NeuralNetworkMLP(capas_arquitectura=[2, 3, 1],
14     tasa_aprendizaje=0.5)
15
16 # 2. Entrenar durante 2000 épocas
17 print("Iniciando entrenamiento...")
18 mlp.entrenar(X_train, Y_train, epochs=2000)
19 print("Entrenamiento finalizado.")
20
21 # 3. Probar predicciones
22 print("\nPredicciones finales (redondeadas):")
23 predicciones = mlp.predecir(X_train)
24 for i in range(X_train.shape[1]):
25     entrada = X_train[:, i]
26     pred = predicciones[0, i]
27     print(f"Entrada: {entrada} -> Predicción: {pred:.4f} (
28         Redondeado: {int(round(pred))})")

```

8.3. Resumen del Flujo Matemático en Código

- **Matrices Transpuestas (.T):** Son fundamentales. En el backpropagation, para multiplicar gradientes que vienen de una capa $l + 1$ ($shape : (n_{l+1}, 1)$) por activaciones de una capa l ($shape : (n_l, 1)$) y obtener un gradiente de pesos W_l ($shape : (n_l, n_{l+1})$), necesitamos transponer matrices para que las dimensiones coincidan matemáticamente. `np.dot(dZ, A.T)`.
- **np.dot vs *:** `np.dot` realiza la multiplicación de matrices (álgebra lineal). `*` realiza la multiplicación elemento-a-elemento. Se usa `*` cuando aplicamos la derivada de la sigmoide: `error_propagado * sigmoide_derivada(A_actual)`.
- **Regla de la Cadena Manual:** El bucle `for` en `backward` que recorre las capas hacia atrás (`range(num_capas - 1, 0, -1)`) está aplicando la regla

de la cadena paso a paso, acumulando el impacto del gradiente desde la salida.

Bibliografía

- [1] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. (Referencia principal para las bases probabilísticas, regresión lineal, paramétrica y K-Means).
- [2] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer. (Fundamentos matemáticos detallados sobre métodos de ensamble, Árboles de decisión, SVM y análisis de componentes principales).
- [3] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. (Texto definitivo para el desarrollo de la arquitectura de redes neuronales, el algoritmo de retropropagación y la optimización basada en gradientes).
- [4] Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press. (Referencia esencial para los enfoques de aprendizaje supervisado y no supervisado desde la teoría de la probabilidad y estadística).
- [5] Nielsen, M. A. (2015). *Neural Networks and Deep Learning*. Determination Press. (Fuente didáctica clave para la implementación paso a paso de perceptrones multicapa y la derivación de la regla de la cadena aplicada al backpropagation).
- [6] Giron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media. (Referencia para la estructuración taxonómica moderna de los algoritmos de Machine Learning y su aplicación práctica).